
Summary of Part II and Further Reading

This chapter – using a concrete univariate integral as a guidepost – built intuition about the connection between inference and computation. This led to some conceptual insights:

Classical methods can be re-framed as probabilistic. Certain elementary numerical methods can be derived precisely as maximum a posteriori estimates under equally elementary probabilistic models. More specifically, we saw that the trapezoidal quadrature rule arises as the MAP estimate under a Wiener process prior on the integrand.

Design policies arise naturally. The placement of evaluation nodes on a regular grid arose as the choice that minimises posterior variance over the integral.

Prior knowledge begets bespoke methods. We encountered the general recipe for the construction of a probabilistic numerical method. First, define a *joint generative* model (a prior and a likelihood) for the latent, intractable, quantity and the “observable” (computable) quantities. Next, construct an action rule through specifying a loss function, determining which computations to perform. For example, in integration, by aiming to reduce the variance of the posterior. By encoding knowledge in a prior measure and goals in a loss function, one can construct customised numerical methods, tailored to a specific task.

Meaningful error measures require good priors. The choice of prior determines the scale of the posterior, and hence influences

how representative the posterior variance is as an error measure. Hierarchical inference allows the model class to be adapted at runtime to *calibrate* the posterior uncertainty. This calibration is epistemic – it relies on the assumption that the “unobserved” parts of the problem (here, the integrand) are subjectively similar to those already observed.

Probabilistic Numerics can be fast. Assigning a posterior probability distribution to a numerical task need not be significantly more expensive than “classic” point estimation. Even uncertainty calibration may be achieved at almost *the same cost* as the classical point estimate. Moreover, we have seen that some probabilistic numerical algorithms, like *wsabi*, are legitimately faster than alternatives. This leads us to frame the computation spent on probabilistic modelling as an *investment*, rather than overhead.

Imposed randomness can be harmful. While Monte Carlo estimators can be elegant from an analytic perspective, computationally, it can be counter-productive to artificially introduce randomness into an otherwise deterministic problem. Monte Carlo estimators can at best converge with stochastic rate, even on extremely regular, simple, estimation problems, like the one used in this chapter.

These observations mirror aspects of inference well known in statistics. In many ways, numerical computation and statistics share the same challenges: which model classes lead to good estimation performance, at acceptable computational cost? To which degree can the error, the deviation of the estimator from the true value, be estimated? But there is also a crucial difference between numerical tasks and statistical inference from physical data sources. In stark contrast to “real-world” statistics, e.g. in the social sciences or medicine, a numerical task is defined not through a vague concept, but in a fully descriptive formal language – the programming language encoding the task. It is thus possible, at least in principle, for the computer itself to check the validity of prior assumptions, and thus to choose among available numerical methods with differing prior measures. This prospect is the subject of contemporary research.

Software

At the time of writing, the open-source *emukit* library¹ provides a package for Bayesian quadrature with a number of different

¹Paley et al. (2019), available at [emukit.github.io](https://github.com/emukit/emukit)

acquisition functions, including for WSABI-L. `Emukit` is an outer-loop package in the sense that it provides the active learning loop, but the surrogate model has to be coded and wrapped by the user into an `emukit` interface. In contrast, the `ProbNum` library² which focuses on the lower numerical level, additionally provides surrogate model functionality and contains a Bayesian quadrature component under active development.

² Code at probnum.org. See the corresponding publication by Wenger et al. (2021).

Further Reading

The main purpose of this chapter was to introduce core notions of probabilistic numerical computation in the simple setting of univariate integration. However, numerical integration poses a field of its own right, and remains the subject of intense study to the present day. Readers interested in deeper insights may find the following notes helpful.

Quasi-Monte Carlo

Quasi-Monte Carlo (QMC) is the name used to describe integration rules that are based on unit weights (i.e. of the algebraic form of the Monte Carlo estimator of Eq. (9.4)), but which use carefully crafted designs of low discrepancy. The error and convergence analysis of QMC methods is related to that of Bayesian quadrature, and has had some success in achieving appealing convergence rates even in the high-dimensional setting. For more on QMC and its analysis using RKHS hypothesis spaces can be found in a review by Dick et al.³

³ Dick, Kuo, and Sloan (2013)

Closely Related Concepts

Related to Bayesian quadrature are topics including “kernel herding”⁴ and kernel quadrature, which themselves are related to other numerical approximation methods.⁵ A subtle difference between Bayesian quadrature and kernel quadrature is that the latter often allows for the kernel to change as a function of the observation nodes (in particular, to shrink as their number grows). Doing so poses conceptual problems in the Bayesian framework: it is not compatible with the philosophical notion of a prior; but more practically, this adaptation of the kernel amounts to a continuous relaxation of the associated probabilistic error estimate (which does not play such a central role in the kernel view).

⁴ Chen, Welling, and Smola (2010); Huszar and Duvenaud (2012).

⁵ Bach, Lacoste-Julien, and Obozinski (2012); Bach (2017).

Convergence Analysis

The theoretical treatment of convergence of Bayesian quadrature has reached a sophistication beyond what can be presented in this chapter. See for example, recent works by Briol et al. (2019), and Kanagawa, Sriperumbudur, and Fukumizu (2020). In particular, both kernel quadrature and Bayesian quadrature using translation-invariant kernels can achieve the optimal worst-case rate for integrands constrained to lie in a Sobolev space, if certain requirements on the distribution of the design points are fulfilled. As already mentioned above, even the more theoretically challenging case of *adaptive* Bayesian quadrature has now been furnished⁶ with results establishing asymptotic consistency (under some loose conditions).

⁶ Kanagawa and Hennig (2019)